

Immersive Reading for Zuntold

Bringing audio and text together

Presented by Haiyu Zhang

Who I Am



Haiyu Zhang

Learning Designer and Developer

- MS, Learning Design & Technology – Stanford University
- BA, Comparative Literature & Information Science – Columbia University
- Previously: Google, Headspace, MIT Media Lab, Stanford Life Design Lab

I build immersive reading experiences because I understand both the technology and the human need behind them – particularly for readers with dyslexia and ADHD.

Introduction

Mission

Zuntold helps young readers engage with books through bibliotherapy

Vision

- Expand impact for students with ADHD and dyslexia
- Opportunity: Immersive reading features strengthen eligibility for accessibility grants such as
 - i4i -Invention for Innovation- grant administered by NIHR (National Institute of Care and Research)

Problem Statement

Zuntold already has great audio and great text. The opportunity is to bring them together into a fully immersive experience designed specifically for readers with dyslexia.

How Can We Support Dyslexic Readers?



Text-to-speech (TTS) & read-aloud

Read-aloud with dual highlight

Simultaneous audio + visual word tracking (sentence highlighted in one color, spoken word in another). Students with learning disabilities scored on par with typical readers for comprehension.

Strong evidence

Speech synthesis for dyslexia

Post-secondary students with dyslexia showed improved reading speed, comprehension, and sustained attention when using speech synthesis during reading tasks (Elkind, via Microsoft IR research).

Strong evidence

Microsoft Immersive Reader

Combines TTS, syllable segmentation, line focus, and customizable display. Used by 23M+ people/month. Shown to improve reading rate, comprehension, attention, word recognition, and subsequent writing quality.

Strong evidence

Pacuilla, H. — Assistive Technology and Adult Literacy: Access and Benefits (2007)

Journal / Microsoft Educator Center Research Library

Students with LD who received bimodal input (audio + text) performed as well on comprehension tests as average-reader controls who only had visual input.

Yeatman, J. et al. — Optimizing text for an individual's visual system (Microsoft IR)

Visual crowding subtype — personalized intervention

Identified a dyslexia subtype involving elevated crowding. Personalized text display (shorter lines, adjusted spacing) improved outcomes significantly for this subtype.

↔ Line length & layout

27% faster reading speed

Shorter line lengths significantly help readers who struggle. A subset of dyslexic readers — particularly those with elevated visual crowding — benefit most.

Strong evidence

Left alignment only

Justified text creates irregular word spacing that makes tracking harder. Consistent left alignment provides a reliable return-sweep point for each line. WCAG and the British Dyslexia Association both recommend this.

Strong evidence

Line spacing

Line height ~1.5× the font size provides the best reading performance for most users (McLeish study, underpins WCAG spacing requirements). Avoids lines "crowding" each other vertically.

Strong evidence

Schneps, M.H. et al. — Shorter Lines Facilitate Reading in Those Who Struggle

Harvard Smithsonian Center for Astrophysics

27% increase in reading speed with shorter line lengths for struggling readers. Linked specifically to those with elevated visual crowding — not a universal effect.

A·B Letter spacing & word spacing

20% easier to read

Optimizing inter-letter spacing can make characters up to 20% easier to read vs defaults (Arditi, 2004). Reduces "crowding" — a core perceptual deficit in many dyslexic readers.

Strong evidence

Extra-large letter spacing (Zorzi et al.)

Landmark PNAS study: increased inter-letter spacing improved reading speed and accuracy for Italian dyslexic children across words, nonwords, and sentences — without training.

Strong evidence

Word spacing — contested

Some studies (Perea et al. 2012) show benefit from wider word spacing. Others find the advantage only holds for individual words, not full reading tasks. Net: widen slightly, but letter spacing matters more.

Moderate / mixed

Zorzi, M. et al. — Extra-large letter spacing improves reading in dyslexia

PNAS, 2012 — doi: 10.1073/pnas.1205566109

Tested 74 Italian children (34 dyslexic). Extra-large spacing reduced errors by ~50% for words and nonwords in dyslexic readers, with no significant cost for typical readers.

syl-la Syllabification & word segmentation

10% comprehension gain

Within-word text segmentation (syllable breaks) improved reading comprehension by 10% in adults (Tai et al., via Microsoft IR). Core feature of Immersive Reader.

Strong evidence

Syllabic bridge hypothesis

Doignon-Camus et al.: building the syllabic bridge from phonological syllables and frequent letter clusters is the first step in learning spelling-to-sound correspondences — foundational for dyslexic readers.

Strong evidence

< EXIT BOOK

Always here for you

?

ACTIVITIESNOTESSETTINGS

🔊 🔍 ⏮ ⏭

Madison and The Bezzies
A New Beginning

It was still raining on Monday morning after a wet weekend stuck indoors with 'the olds' as Holly now called her parents. She'd heard Madison say it once and there was a satisfying frown from Mum when she threw it out at home. It was still raining on Monday morning after a wet weekend stuck indoors with 'the olds' as Holly now called her parents. She'd heard Madison say it once and there was a

READLISTEN

0:13

13:38

1x

⏮ ⏭

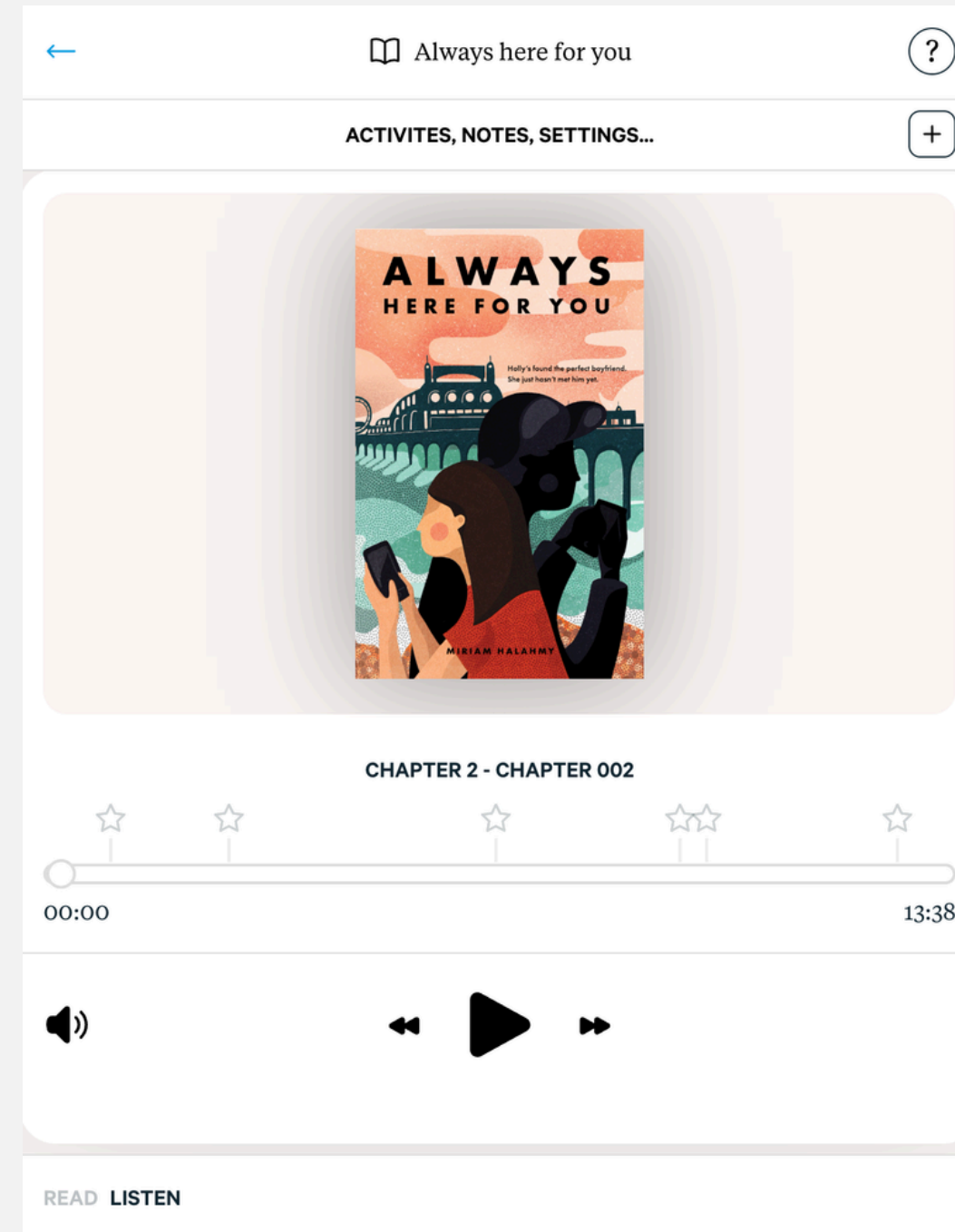
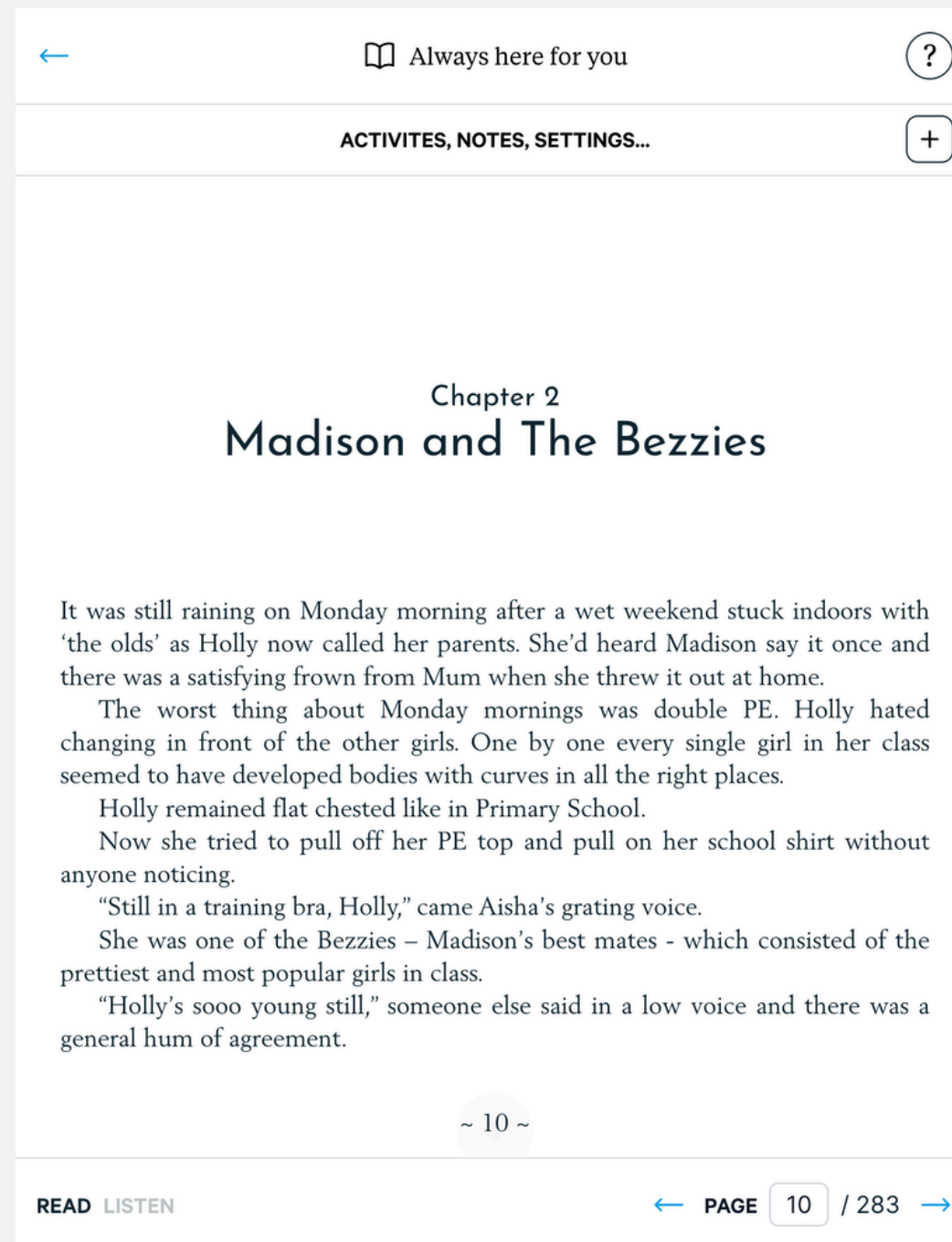
▶

⏮ ⏭

< PAGE 10 / 283 >

Currently

4. fixed typography,
line length, word
spacing



3. Audio player
has no speed
control

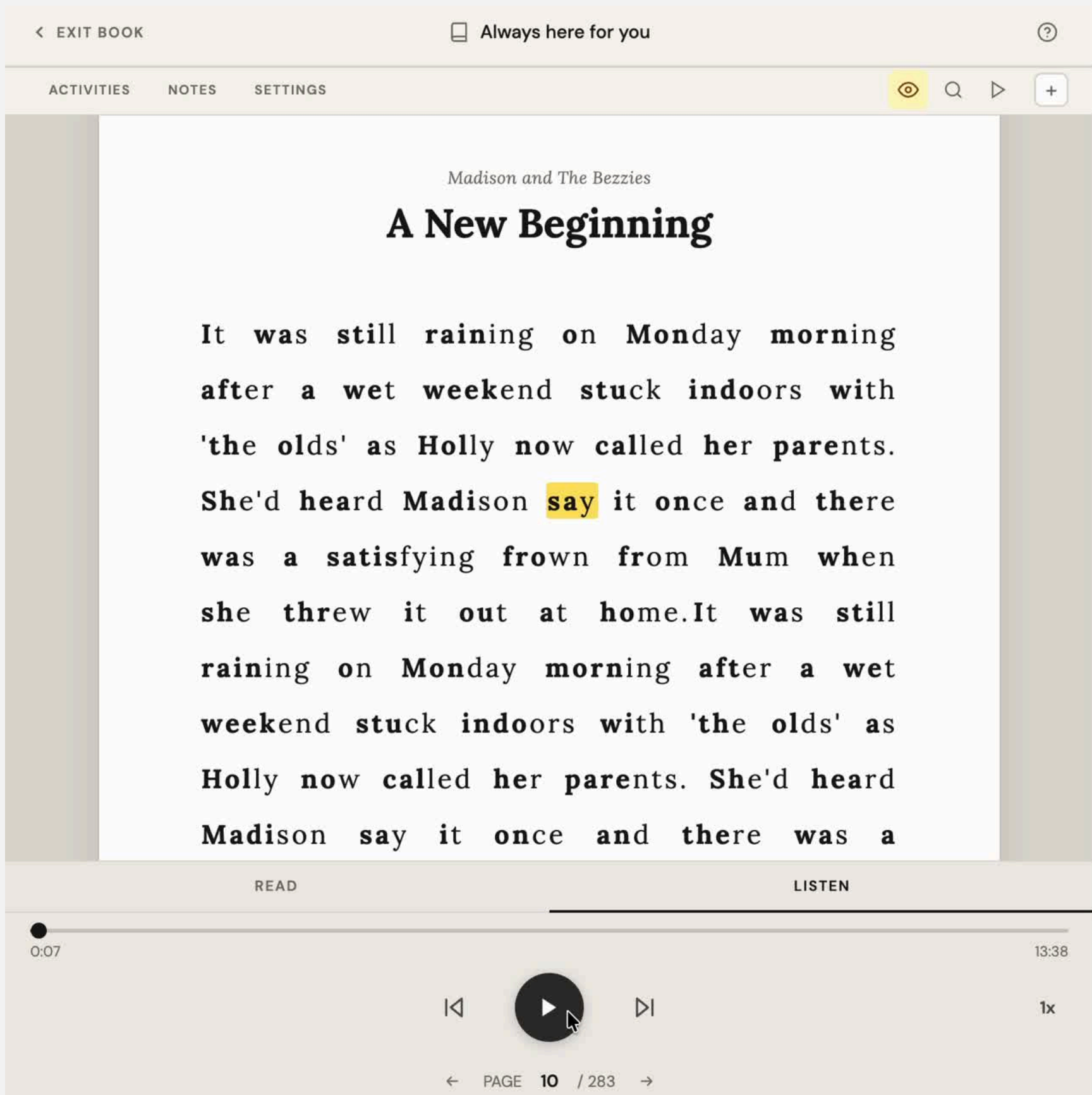
1. READ and LISTEN are currently two separate pages – no unified experience.
2. No word-by-word synchronisation between audio and text

What I built

Frontend features (Alpine component)

Feature	Notes
Word-sync highlight	Updates on <code>audio.timeupdate</code> (~4 Hz)
Click-to-seek	<code></code> click sets <code>audio.currentTime</code>
Bionic reading	Bolds first half of each word
Playback speed	Presets 0.8x–2.5x + fine stepper
Typography panel	Font, size, spacing, line-height (persists locally)
Narration mode	Human voice (fallback: browser TTS)

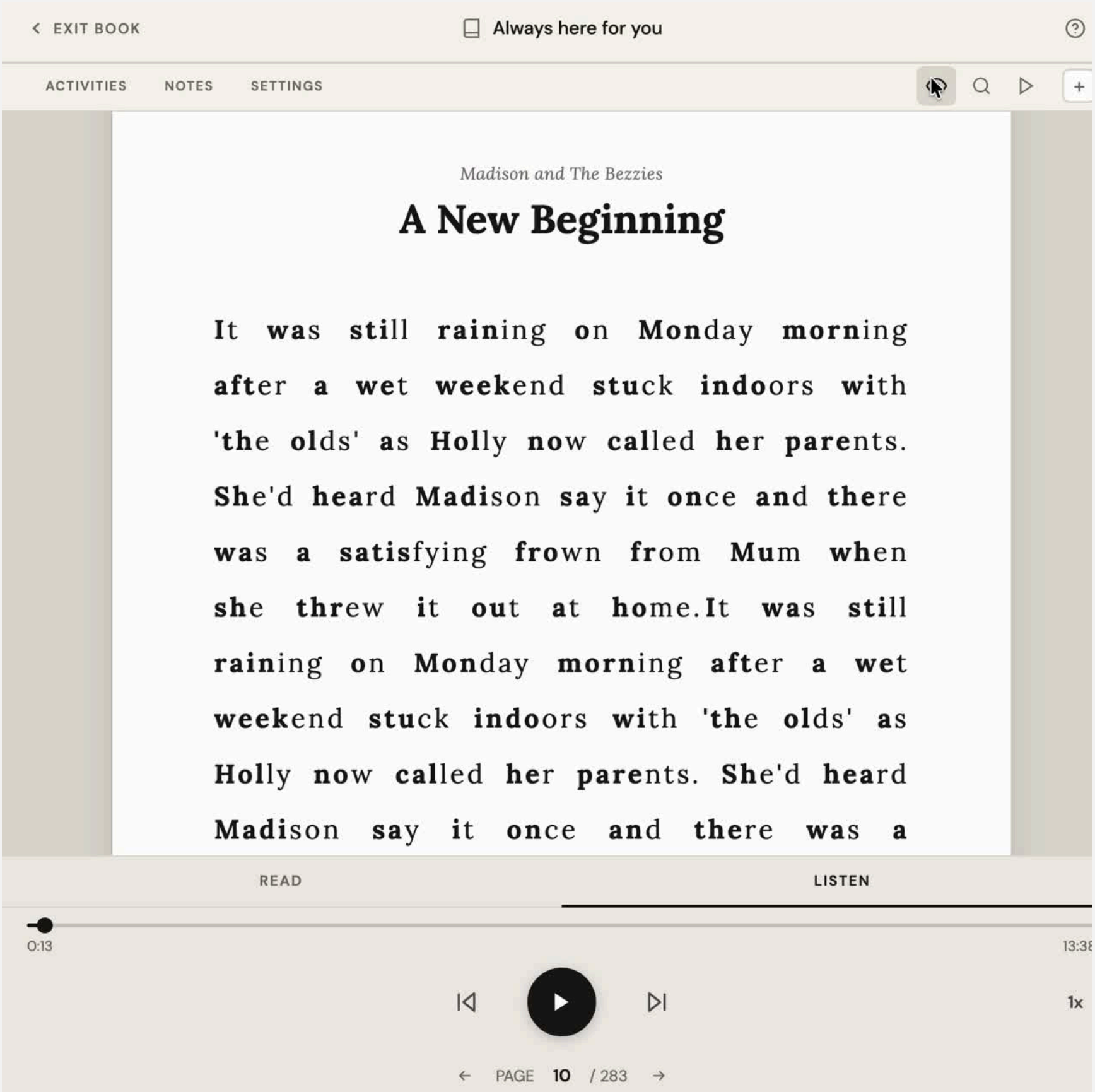
What I built



TTS read-aloud with
real-time word-by-
word highlight

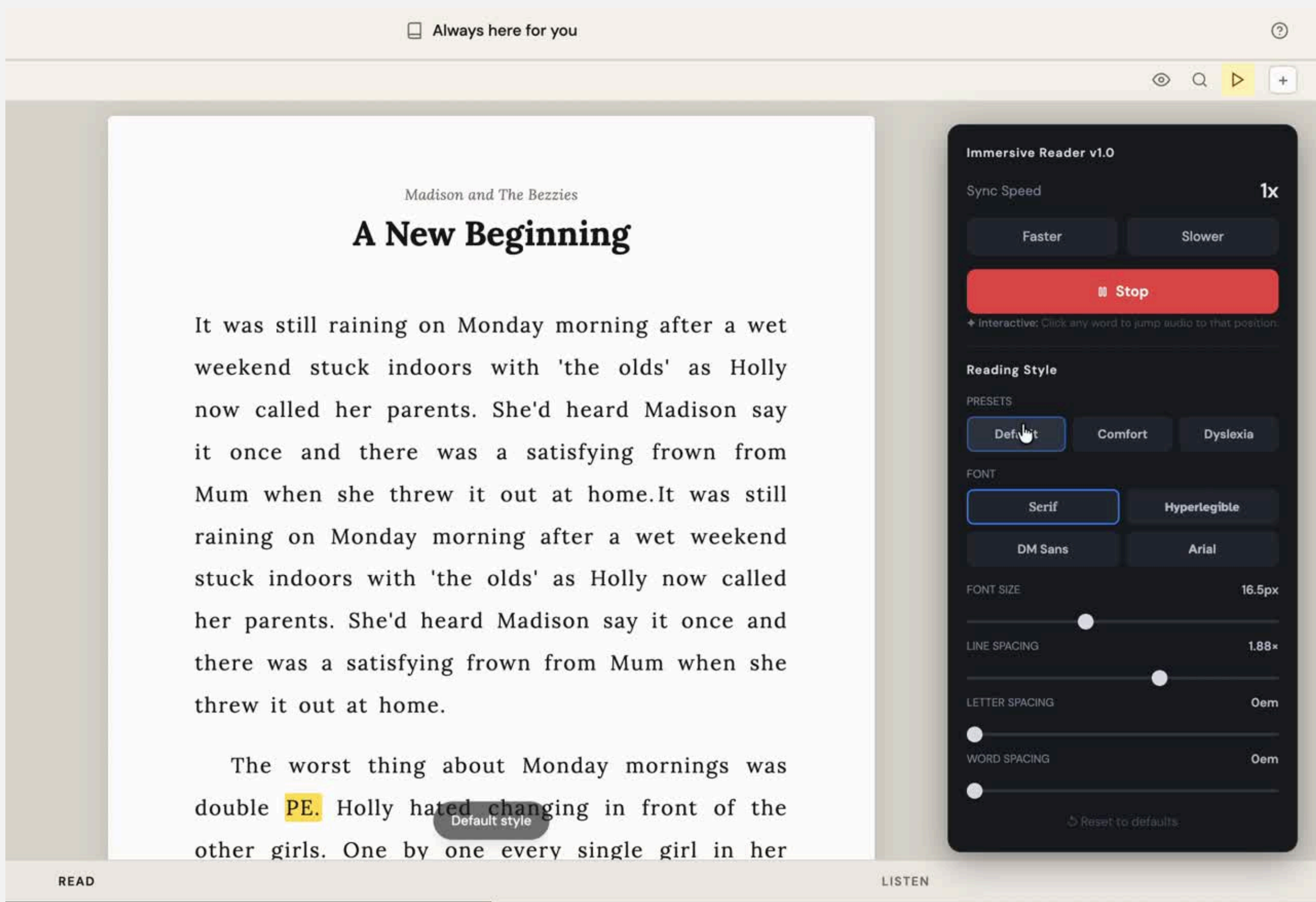
Click any word to jump
playback to that
position

What I built



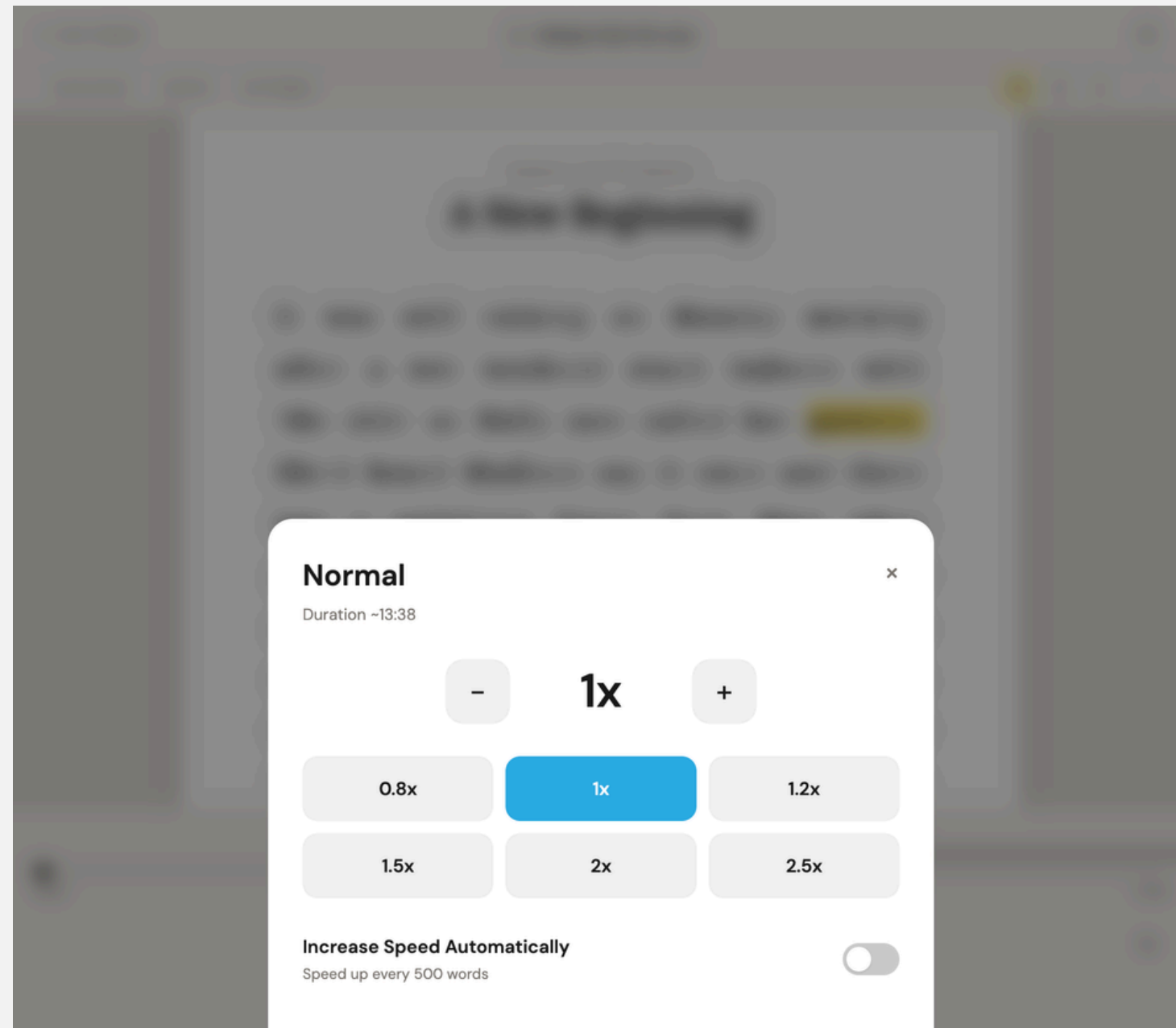
Bionic reading –
bolds first half of
each word to guide
the eye

What I built



Personalisation panel – font, size, line height, letter spacing, dyslexia preset

What I built



Speed control (0.8x–2.5x) + auto-speed mode

Technical Decisions

Stack used

This demo is built with the **same stack as the main project** — no new dependencies:

Tool	Role
Tailwind CSS	All layout and styling (pre-built via CLI — no browser compiler)
Alpine.js	All interactivity via <code>x-data="immersiveReader()"</code>
Web Speech API	Browser-native TTS — no external service needed

The Alpine component is structured to drop straight into `resources/js/` and register as `Alpine.data('immersiveReader', immersiveReader)`.

Book content is hardcoded for this demo — in production it comes from Laravel as `@json($book->paragraphs)`.

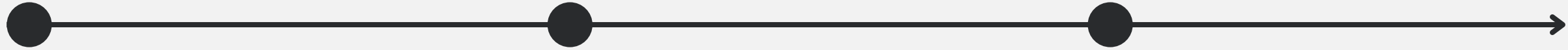
Integration path

How this maps to the main project structure



```
resources/  
  views/reader/immersive.blade.php ← HTML from index.html  
  js/reader.js                     ← immersiveReader() Alpine component  
  css/reader.css                   ← Custom styles (word-span, bionic)  
  
app/Http/Controllers/  
  ReaderController.php             ← Passes book data to view  
  
routes/web.php                     ← GET /book/{id}/read
```

What's Next



PHASE 1

Immersive Reading
TTS (Word Sync,
Click to See),
current UI match

PHASE 2

Biotic Bolding,
Speed Control,
Line Spacing
Interface

PHASE 3

Human Voice
TTS

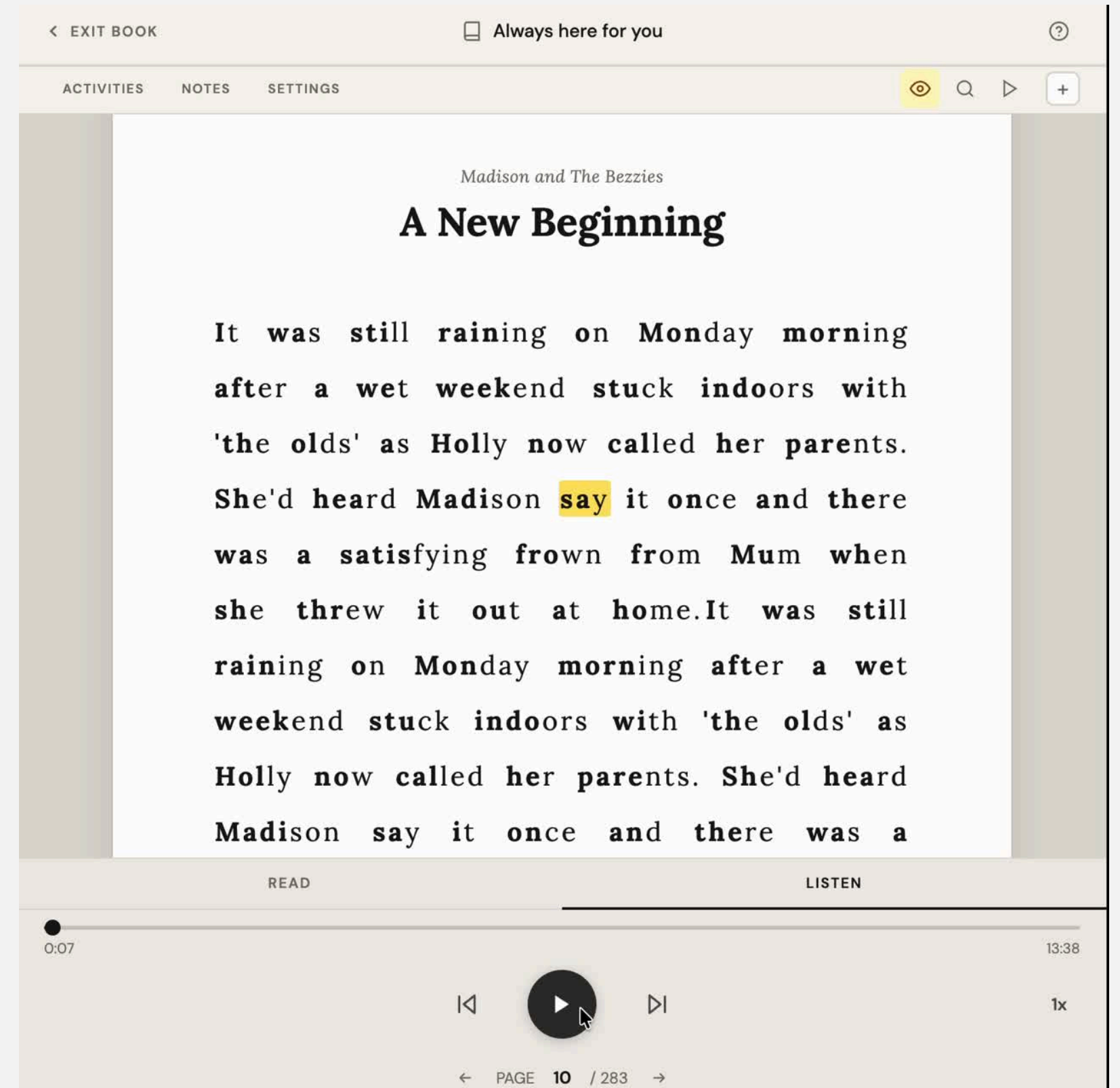


User Testing

- Phase 1** → Small-scale user testing: pilot with existing Zuntold users, collect feedback
- Phase 2** → Formal accessibility testing: recruit dyslexia & ADHD users, record data → this data feeds directly into grant applications
- Phase 3** → Comparative testing: browser TTS voice vs human recordings — which do users prefer?

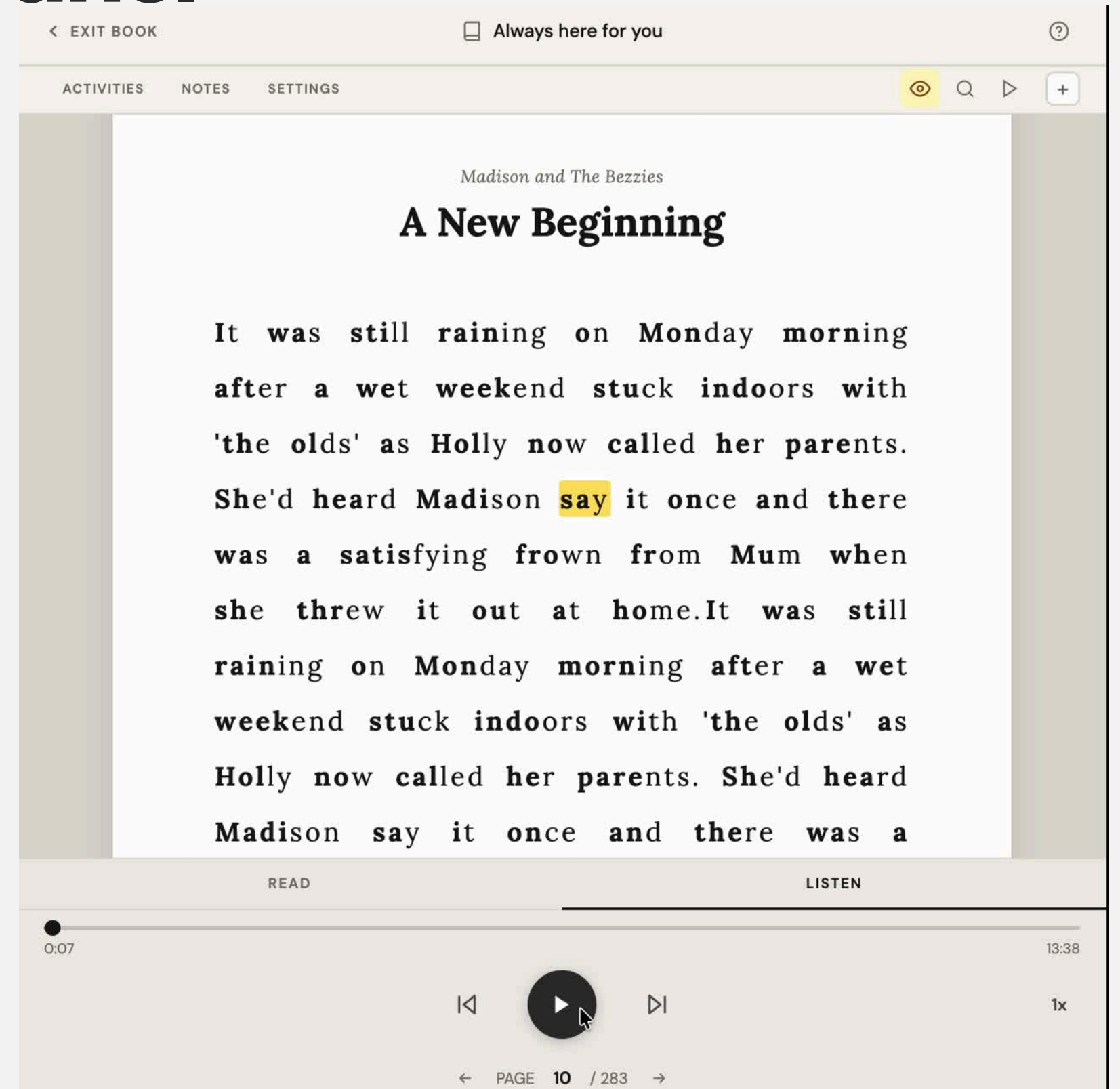
Phase 1 - Immersive Reading TTS

- Connect Alpine component to Laravel — replace hardcoded content with @json(\$book->paragraphs)
- TTS via Web Speech API (browser-native, no cost)



Phase 2 - Dyslexia Friendly Panel

- Bionic reading toggle
- Personalisation panel — font, size, line height, letter spacing
- Speed control



Phase 3 - Human Voice

Phase 1 — available now

Browser TTS (Web Speech API)

- No external service needed
- No cost — completely free
- Voice comes from the user's OS — Siri on Mac, Microsoft Aria on Windows
- Quality is decent but clearly synthetic — not a human voice
- Mac users: sounds like Siri reading a book
- Windows users: sounds like Microsoft Aria

Phase 1 is the prototype

Fully functional immersive experience — word sync, bionic, personalisation all work. Voice is robotic but the feature is real and usable from day one.

How TTS & word sync works

Browser-native Web Speech API — no external service, no cost

BUILDING THE UTTERANCE

```
buildUtterance(fromChar) {  
  // Slice text from current position  
  // so user can resume mid-chapter  
  const u = new SpeechSynthesisUtterance(  
    this.fullText.slice(fromChar)  
  );  
  
  if (this.chosenVoice)  
    u.voice = this.chosenVoice;  
  u.rate = this.speechRate;  
  
  // Fires on every word boundary —  
  // this is what drives the highlight  
  u.onboundary = (e) => {  
    if (e.name !== 'word') return;  
    this.highlightWord(  
      this.wordIdxAtChar(  
        fromChar + e.charIndex  
      )  
    );  
  };  
}
```

PICKING THE BEST AVAILABLE VOICE

```
pickBestVoice() {  
  const voices =  
    window.speechSynthesis.getVoices();  
  
  // Prefer natural-sounding voices —  
  // falls back to any English voice  
  const preferred = [  
    'Samantha', // macOS default  
    'Karen',    // macOS AU  
    'Daniel',   // macOS UK  
    'Microsoft Aria', // Windows  
    'Google UK English Female',  
  ];  
  
  for (const name of preferred) {  
    const v = voices.find(  
      v => v.name.includes(name)  
    );  
    if (v) return v;  
  }  
  
  // Last resort: any English voice  
  return voices.find(  
    v => v.lang.startsWith('en')  
  ) || voices[0];  
}
```

The onboundary event is the key — the browser fires it on every word as it's spoken, giving us a character index we use to highlight the exact word in real time. No external API, no latency, no cost.

Phase 2 — two options for real audio sync

Same end result: human voice + word-by-word highlight. Different tradeoffs.

Option A — recommended

Whisper API — transcribe existing recordings

Keep Zuntold's human voice recordings, add timestamps

How it works	Run each recording through Whisper API → get word-level timestamps automatically
Cost	~\$1–5 one-time for entire library
Ongoing cost	None — timestamps stored in DB
Voice quality	Zuntold's existing human recordings — no change
Risk	Whisper accuracy ~95%+ — occasional manual correction needed
Backend change	Add transcript + timestamps table to DB

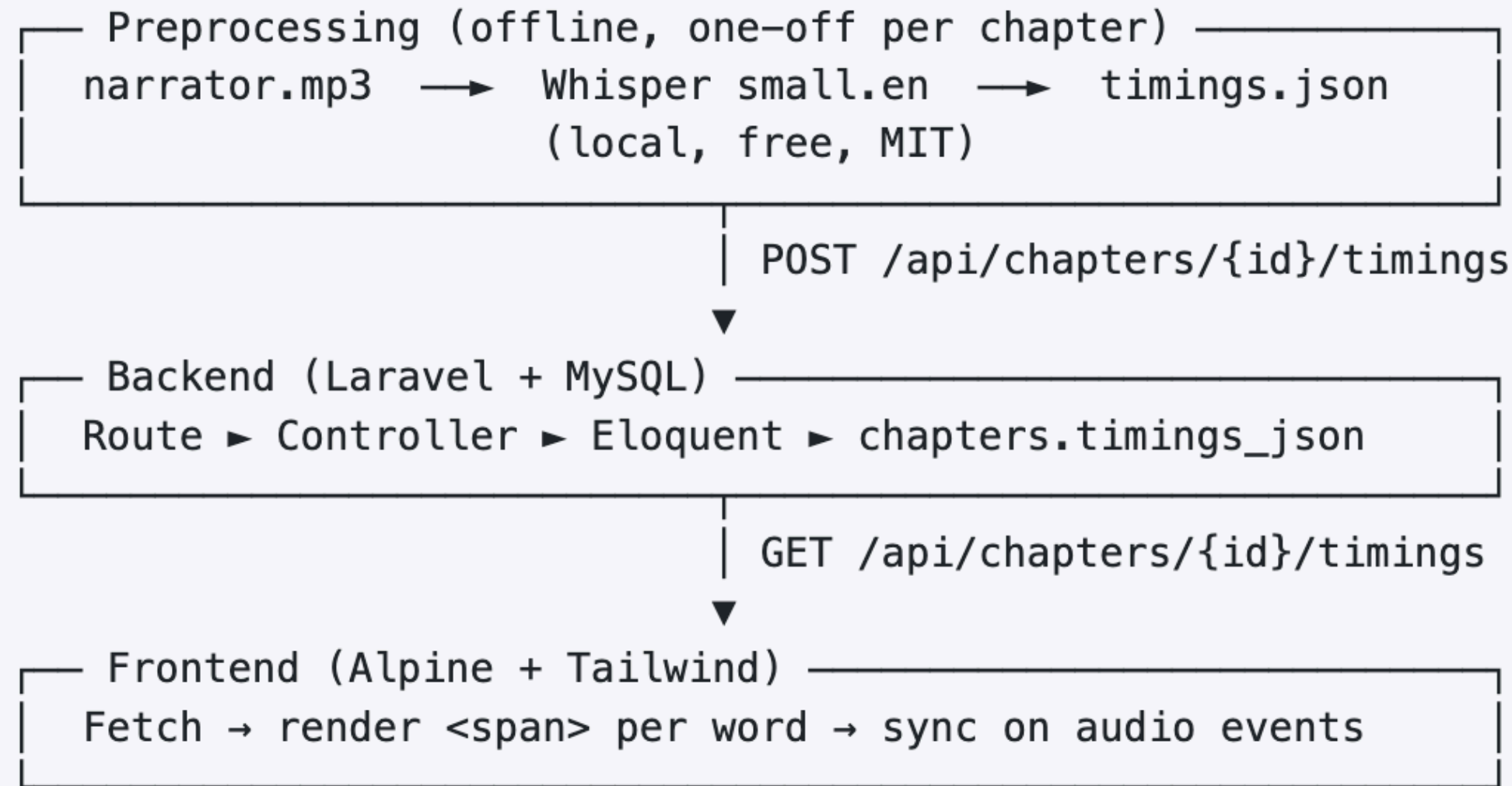
Option B — future consideration

TTS API — regenerate audio with timestamps

Replace recordings with AI voice that has built-in sync

How it works	ElevenLabs / Google TTS generates audio + timestamps in one call
Cost	ElevenLabs from \$5/month, Google TTS ~\$1–2 per book
Ongoing cost	Per new book added to library
Voice quality	High quality AI voice — but replaces human recordings
Risk	Users may notice voice change — human warmth is part of Zuntold's brand
Backend change	New audio pipeline + storage

Architecture



Key design choice: Whisper runs only at upload time. The runtime path (what end users hit) is pure PHP + MySQL — no Python needed on production servers.

How I Can Help

- Own the feature implementation
- Bridge design and engineering
- Evidence-based decisions backed by research
- Run the Whisper transcription pipeline
- Design and interpret user testing sessions



Where I Need Your Feedback

- Your feedback on the approach — does Phase 1 and 2 make sense given your current priorities and timeline?
- Your call on Phase 3 — Whisper API setup touches the backend, so I'd want to work through that together rather than assume
- If it seems right, it would be great if we can have a tech meeting before I start for things like
 - Github repo access
 - Branching & pr standard
 - Current audio storage
 - ...



Thanks! Q&A

**I've built this to fit your stack — but you know the codebase better than anyone.
I'd love to understand how you'd approach the integration together**

- Are there technical constraints I haven't considered?
- What does the ideal collaboration look like from your side?

